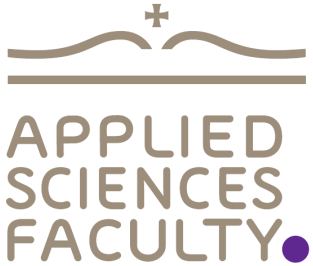
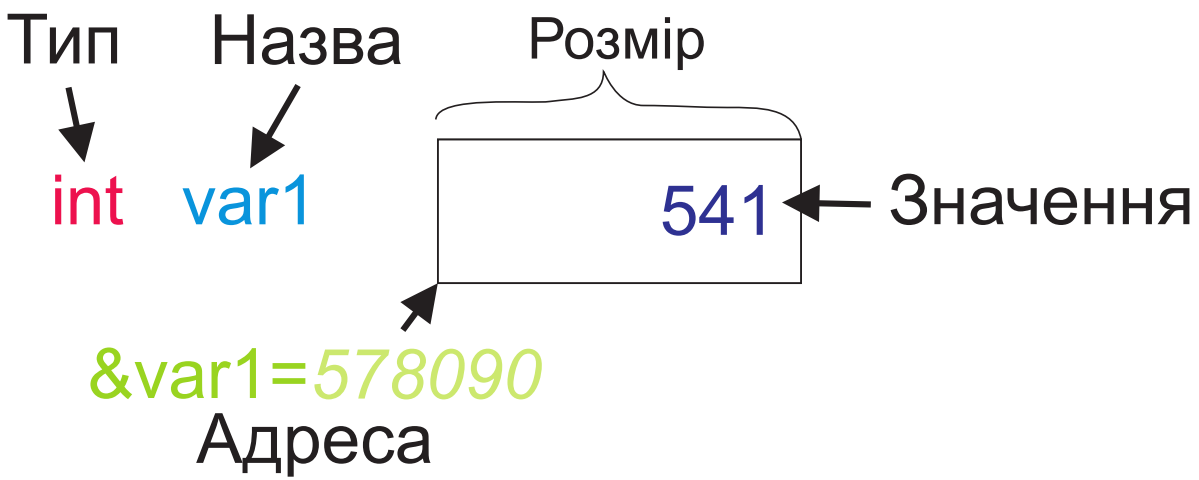
C++ — лекція 1. Змінні, типи, оголошення, константи.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Змінні та їх типи



Ідентифікатори

- Ім'я змінної повинно бути коректним **ідентифікатором**.
- Ідентифікатор повинен починатися з літери і містити виключно літери та цифри. Літерами вважаються символи англійського алфавіту, a-z, A-Z та символ підкреслення[1].
- **Великі та малі літери — вважаються різними**, `test`, `Test` і `TEST` — різні ідентифікатори.
- Ідентифікатори не повинні починатися з підкреслень або містити два підкреслення підряд[2].
- ◇ Строго кажучи, ситуація трішки складніша і різноманітніша, ніж описано вище. Однак, рекомендації вище, хоча їх і можна порушувати за певних умова — найбільш безпечні. Детальніше див. `http://en.cppreference.com/w/cpp/language/identifiers` та `https://devblogs.microsoft.com/oldnewthing/20230109-00/?p=107685` – "On leading underscores and names reserved by the C and C++ languages".

[1] Формально, дозволено багато символів Unicode і MSVC (частково?) підтримує цю можливість. Однак, не робіть так – не користуйтеся ними. Див. також `https://en.cppreference.com/w/cpp/language/identifiers`.

[2] Це не є синтаксичною помилкою, однак такі імена вважаються зарезервованими для потреб компілятора та стандартної бібліотеки.

- Тема ідентифікаторів з підкресленням ніби того не вартує, але дуже люблять програмісти ці підкреслення, тому подивимося детальніше. Стандарт стверджує:
  - Ідентифікатори з двома підкресленнями підряд, розташованими де завгодно, зарезервовані[3].
  - Ідентифікатори, що починаються із підкреслення, після якого – велика літера, зарезервовані.
  - Всі ідентифікатори, що починаються із підкреслення, зарезервовані в глобальному просторі імен.
- Зарезервований – означає, що стандартна бібліотека може визначати такі імена, зокрема і за допомогою #define, для своїх потреб, компілятор може визначати такі символи сам по собі, механізм заплутування імен може створювати їх в процесі своєї роботи.
- Використання зарезервованих ідентифікаторів – невизначена поведінка.
- Заголовки WinAPI порушують іноді ці правила – MSVC і MinGW змушені це враховувати. Але не слід повторювати помилки, зроблені в 80-х Microsoft, які зберігаються тепер для зворотної сумісності.
- C, за відсутності просторів імен, резервує цілу множину імен для зовнішнього зв'язування – так можна називати локальні змінні, типи, члени перерахувань, статичні глобальні змінні та функції, але не глобальні змінні та функції, з якими можливе лінкування з інших об'єктних файлів.
  - При тому, C має право ці імена заміняти на макроси – тому, якщо їх таки потрібно використати, слід або робити #undef спершу – якщо оголошені, або брати в дужки ці ідентифікатори.
  - Детальний список є в розділі стандарту 7.31: "Future library directions".
  - До прикладу, поміж них – is*, to*, str*, тощо, де зірочка означає довільні припустимі символи.

Зарезервовані ідентифікатори — 1

- Для ідентифікаторів не можна використовувати зарезервовані, ключові, слова. В C++98/03 це:

```
asm, auto, bool, break, case, catch, char, class, const, const_cast, continue,
default, delete, do, double, dynamic_cast, else, enum, explicit, export,
extern, false, float, for, friend, goto, if, inline, int, long, mutable,
namespace, new, operator, private, protected, public, register,
reinterpret_cast, return, short, signed, sizeof, static, static_cast, struct,
switch, template, this, throw, true, try, typedef, typeid, typename, union,
unsigned, using, virtual, void, volatile, wchar_t, while
```

- C++11 додав[4]:

```
alignas, alignof, auto[5], char16_t, char32_t, decltype, noexcept, nullptr,
register[6], static_assert, thread_local.
```

- C++20:

```
char8_t, concept, consteval, constinit, co_await, co_return, co_yield,
requires.
```

- Подальші оновлення – див. `https://en.cppreference.com/w/cpp/keyword`.

---

[4] Крім того, деякі із перерахованих вище, набули в C++11 додаткових функцій — такі як class чи default. А extern залишилося зарезервованим, але без використання.

[5] Змінило значення в C++11.

[6] Змінило значення в C++17.

Зарезервовані ідентифікатори — 2

- Крім того, за певних умов зарезервованим є додаткові імена. В C++98/03:

`and, and_eq, bitand, bitor, compl, not, not_eq, or, or_eq, xor, xor_eq.`

- В C++11:

`override, final.`

- В C++20:

`import, module.`

- В майбутньому слід бути готовим до того, що наступні слова стануть зарезервованими (використано в різноманітних TS):

`concept, import, module, requires, synchronized, transaction_safe, transaction_safe_dynamic`

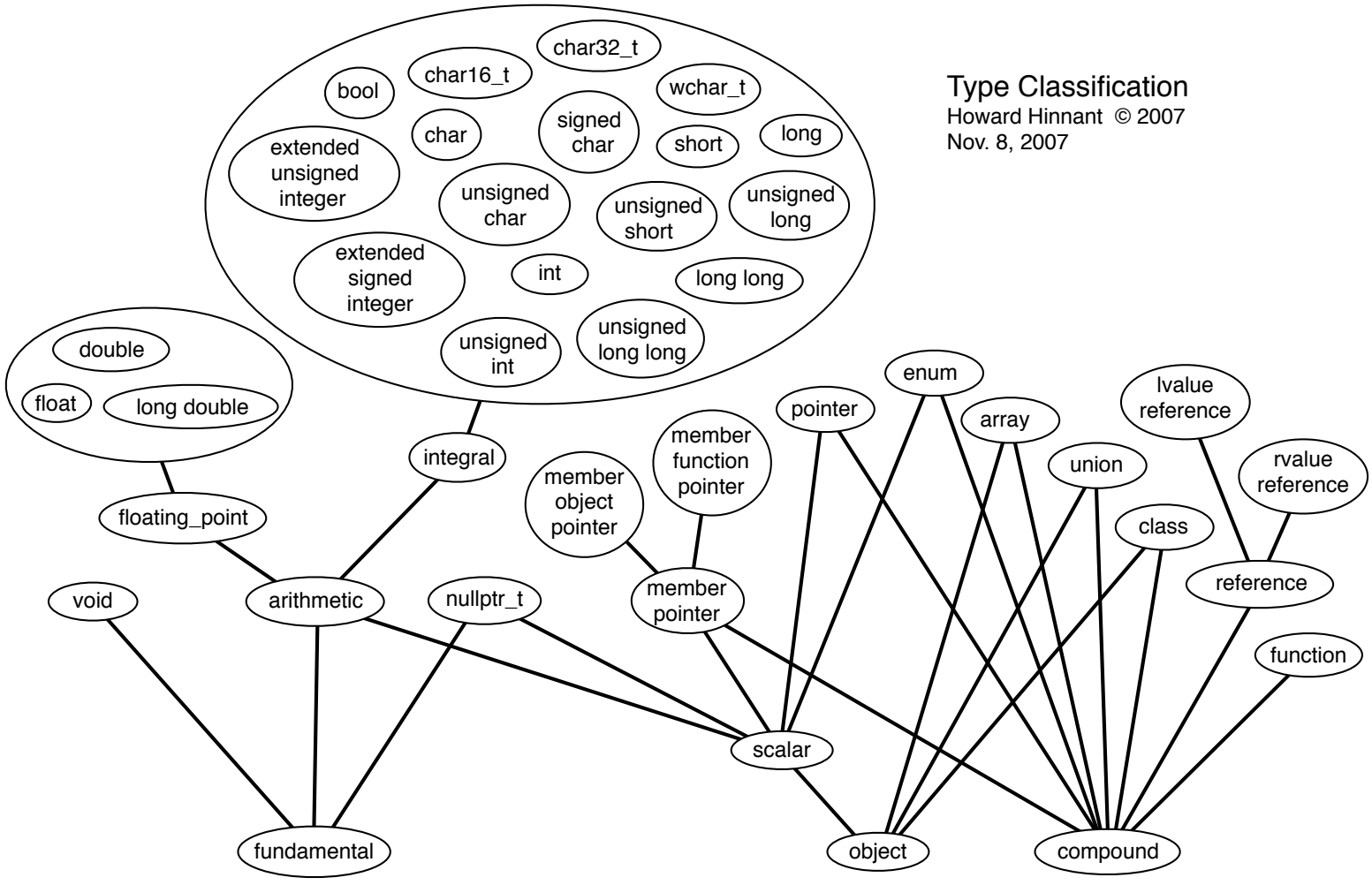
- posix зарезервовано (C++11) як ім'я для простору імен.
- Детальніше див. `http://en.cppreference.com/w/cpp/keyword`.

Типи

- Фундаментальні:
  - Логічний тип, **bool**.
  - Цілі типи, наприклад **int**.
  - Спеціальним частковим випадком цілих типів є символьні типи, наприклад **char**.
  - Типи з рухомою комою, наприклад **double**.
- Тип void, який вказує на відсутність інформації про тип.
- Перерахування, enum.
- Складені типи:
  - Вказівники, наприклад **int***, **char***.
  - Масиви, наприклад **double[]**.
  - Посилання, наприклад **int&**.
  - Структури та класи.
- Логічний, символьні та цілі типи називають інтегральними.
- Інтегральні типи, разом із типами з рухомою комою, називають арифметичними типами.
- Перерахування, структури та класи — типи, визначені користувачем. Решту типів називають вбудованими.

Ієрархія типів

- Зображення Howard Hinnant, взято тут.



Тестування типів

- C++11 додав інструменти визначення природи типу: is_fundamental, (із подальшою деталізацією: is_void, is_null_pointer, is_arithmetic, is_floating_point, is_integral), is_compound, (is_lvalue_reference, is_rvalue_reference, is_pointer, is_member_pointer, is_member_object_pointer, is_member_function_pointer, is_array, is_function, is_enum, is_class, is_union).
- Додатково можна перевіряти: is_object – все, що не функція, не посилання і не void; is_scalar, is_literal_type – типи, які можуть використовуватися із constexpr, is_pod, is_trivial.
- Оголошені вони в <type_traits>.
- Використання:

```
std::cout << std::is_arithmetic<int>::value << std::endl;
```

- Як видно, це не дуже зручно – громіздко. C++17 додав ряд синонімів:

```
template< class T >
inline constexpr bool is_arithmetic_v
                          = is_arithmetic<T>::value;
```

які можна використовувати так:

```
std::cout << std::is_arithmetic_v<int> << std::endl;
```

Інтегральні типи — типи цілих чисел

Крім булівського, бувають у трьох варіантах — варіант по замовчуванню для системи, знаковий та беззнаковий:

- char, signed char, unsigned char — символьні типи, одиниця відліку розміру інших типів.
- short, (синонім — short int), signed short, unsigned short — короткий цілий тип.
- int, signed int, unsigned int — базовий цілий тип, той, з яким найбільш ефективно працює процесор, для якого програмуємо.
- long, signed long, unsigned long — довгий цілий тип.
- C99 ввів також long long, signed long long, unsigned long long, з C++11 вони є і в C++.
- signed, unsigned та short, long, long long розглядаються як модифікатори, порядок яких довільний, тому "unsigned long long int" і "long int unsigned long" – синоніми.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/types`.

Розмір типів

Стандарт **не нав'язує** конкретні, фіксовані, розміри типів. На різних платформах їх величина різна. Стандарт вимагає лише:

- char — мінімум 8 біт,
- short — мінімум 16 біт,
- long — мінімум 32 біти.
- long long (C++11) — мінімум 64 біти.

та

sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long).

Додатково, стандарт POSIX вимагає, щоб char був розміром рівно 8 біт. Хоча, існують архітектури з іншим розміром char[7].

Для визначення розміру змінної чи типу, використовується оператор sizeof:

```
size_t s1 = sizeof(int);
int x;
size_t s2 = sizeof(x);
```

`size_t` — тип, достатньо великий, щоб вмістити розмір будь-якого об'єкту, який може існувати в програмі.

[7] Каюся, не вдалося знайти конкретного прикладу...

Характеристики floating-point типів

Через особливість представлення чисел з рухомою крапкою, floating-point типи володіють наступними особливостями:

- Діапазони додатних та від'ємних чисел рівні.
- Крім найбільшого (в світлі попереднього пункту — за модулем), значення, важливим є також мінімальне за модулем, відмінне від нуля, та число, найближче до 1.0 але відмінне від нього, яке можна представити цим типом — іноді цю величину називають епсилон (epsilon).

Основні фундаментальні типи – MSVC

Розмір та діапазон вказані для компілятора C++ із Visual Studio

Тип	Розмір, байт	Допустимі значення
bool	1	true або false
short	2	від -32 768 до 32 767
unsigned short	2	від 0 до 65 535
signed short	2	від -32 768 до 32 767
int	4	від - 2 147 483 648 до 2 147 483 647
unsigned int	4	від 0 до 4 294 967 295
signed int	4	від - 2 147 483 648 до 2 147 483 647
long int	4	від - 2 147 483 648 до 2 147 483 647
unsigned long int	4	від 0 до 4 294 967 295
signed long int	4	від - 2 147 483 648 до 2 147 483 647
long long int	8	від -2^{63} до $2^{63}-1$
unsigned long long int	8	від 0 до 2^{64}
signed long long int	8	від -2^{63} до $2^{63}-1$
char	1	літери з кодами від 0 до 255.
wchar_t	2	"широкий" символьний тип [8].
float	4	$\pm$ від $1.2 \cdot 10^{-38}$ до $3.4 \cdot 10^{+38}$, $\epsilon = 1.2 \cdot 10^{-7}$, digits: 6
double	8	$\pm$ від $2.2 \cdot 10^{-308}$ до $1.8 \cdot 10^{+308}$, $\epsilon = 2.2 \cdot 10^{-16}$, digits: 15
long double	8[9]	Те ж, що і для double

[8] Детальніше див. http://www.joelonsoftware.com/articles/Unicode.html і дискусію тут: http://stackoverflow.com/questions/1613494/why-was-wchar-t-invented

[9] Апаратура x86 підтримує розширений, 10-байтовий тип, який деякими компіляторами використовується в ролі long double, але VS його ігнорує.

Основні фундаментальні типи — GCC4+/Linux/x86-64

Тип	Розмір, байт	Допустимі значення
bool	1	true або false
short	2	від -32 768 до 32 767
unsigned short	2	від 0 до 65 535
signed short	2	від -32 768 до 32 767
int	4	від - 2 147 483 648 до 2 147 483 647
unsigned int	4	від 0 до 4 294 967 295
signed int	4	від - 2 147 483 648 до 2 147 483 647
long int	8	від - 9 223 372 036 854 775 808$= -2^{63}$ до 9 223 372 036 854 775 807$= 2^{63}-1$
unsigned long int	8	від 0 до 18 446 744 073 709 551 616 $= 2^{64}$
signed long int	8	від -2^{63} до $2^{63}-1$
long long int	8	те ж, що і вище, від -2^{63} до $2^{63}-1$
unsigned long long int	8	від 0 до 2^{64}
signed long long int	8	від -2^{63} до $2^{63}-1$
char	1	символи з кодами від 0 до 255.
wchar_t	4	літери з кодами від 0 до 255.
float	4	$\pm$ від $1.2 \cdot 10^{-38}$ до $3.4 \cdot 10^{+38}$, $\epsilon = 1.2 \cdot 10^{-7}$, digits: 6
double	8	$\pm$ від $2.2 \cdot 10^{-308}$ до $1.8 \cdot 10^{+308}$, $\epsilon = 2.2 \cdot 10^{-16}$, digits: 15
long double	16[10]	$\pm$ від $3.4 \cdot 10^{-4932}$ до $1.2 \cdot 10^{+4932}$, $\epsilon = 1.1 \cdot 10^{-19}$, digits: 18

[10] Апаратура x86 підтримує розширений, 10-байтовий тип, розмір 16 для вирівнювання на границю параграфа, 10h.

Інші варіанти для x86-сумісних компіляторів (за даними Agner Fog)

segment word size	16 bit			32 bit						64 bit			
compiler	Microsoft	Borland	Watcom	Microsoft	Intel Windows	Borland	Watcom	Gnu v.3.x	Intel Linux	Microsoft	Intel Windows	Gnu	Intel Linux
bool	2	1	1	1	1	1	1	1	1	1	1	1	1
char	1	1	1	1	1	1	1	1	1	1	1	1	1
wchar_t		2		2	2	2	2	2	2	2	2	4	4
short int	2	2	2	2	2	2	2	2	2	2	2	2	2
int	2	2	2	4	4	4	4	4	4	4	4	4	4
long int	4	4	4	4	4	4	4	4	4	4	4	8	8
__int64				8	8			8	8	8	8	8	8
enum	2	2	1	4	4	4	4	4	4	4	4	4	4
float	4	4	4	4	4	4	4	4	4	4	4	4	4
double	8	8	8	8	8	8	8	8	8	8	8	8	8
long double	10	10	8	8	16	10	8	12	12	8	16	16	16
__m64				8	8				8		8	8	8
__m128				16	16				16	16	16	16	16
__m256					32				32		32		32
pointer	2	2	2	4	4	4	4	4	4	8	8	8	8
far pointer	4	4	4										
function pointer	2	2	2	4	4	4	4	4	4	8	8	8	8
data member pointer (min)	2	4	6	4	4	8	4	4	4	4	4	8	8
data member pointer (max)		4	6	12	12	8	12	4	4	12	12	8	8
member function pointer (min)	2	12	6	4	4	12	4	8	8	8	8	16	16

В ширшому контексті:

short	int	long	ptr	long long	Label		Examples
...	16	...	16	...	IP16		PDP-11 Unix (1973)
16	16	32	16	...	IP16L32		PDP-11 Unix (1977); multiple instructions for long
16	16	32	32	...	I16LP32		MC68000 (1982); Apple Macintosh 68K; Microsoft operating systems (plus extras for x86 segments)
16	32	32	32	...	ILP32		IBM 370; VAX Unix; many workstations
16	32	32	32	64	ILP32LL or ILP32LL64		**Microsoft Win32;** Amdahl; Convex; 1990 Unix systems; Like IP16L32, for same reason; multiple instructions for long long
16	32	32	64	64	LLP64 or IL32LLP64 or P64	64-bit systems	**Microsoft Win64** (X64 / IA64)
16	32	64	64	64	LP64 or I32LP64		**Most Unix systems** (Linux, Solaris, DEC OSF/1 Alpha, SGI Irix, HP UX 11)
16	64	64	64	64	ILP64		HAL; logical analog of ILP32
64	64	64	64	64	SILP64		UNICOS

Джерело: `https://www.viva64.com/en/t/0012/`

- "A taxonomy of C++ types".

Memsize-типи

- **size_t** — тип для розміру будь-якого об'єкту програми в байтах. Синонім для одного із фундаментальних **беззнакових** цілих типів[11]. Зокрема, sizeof повертає саме його, а new — його приймає.
- **ptrdiff_t** — тип представлення віддалі між двома вказівниками. Тип різниці вказівників. Є синонімом одного із фундаментальних знакових типів.

Увага! `size_t` та `ptrdiff_t` важливі не тільки тоді, коли ви працюєте із кількістю даних, більшою, ніж поміщається в int[12]! Див. приклади тут: `http://www.viva64.com/en/l/0013/`.

Оглядова стаття: `https://medium.com/@CPP_Coder/about-size-t-and-ptrdiff-t-a1654234d842`.

[11] Але для різних систем йому відповідають різні фундаментальні типи!

[12] Звична ситуація на x86_64 платформах, скажімо.

std::byte – C++17

- C++17 додав тип байт, std::byte (з <cstddef>):

  enum class byte : **unsigned char** {} ;

  - Його ціль – вираження ідеї, власне, байта – на противагу (unsigned) char, які завжди мали подвійне використання.
  - std::byte не є арифметичним типом і не є символьним типом.
  - Для нього означені хіба побітові операції та функція std::to_integer().

"Конкретизовані" цілі типи з <cstdint> – C++11

- intmax_t, uintmax_t — знаковий та беззнаковий варіанти найбільшого (найбільш ємкого) цілочисельного типу[13].
- int8_t, uint8_t; int16_t, uint16_t; int32_t, uint32_t; int64_t, uint64_t — знакові та беззнакові цілі типи величиною, відповідно 1, 2, 4, 8 байт (8, 16, 32, 64 біти). Означені, лише якщо для даного компілятора відповідні типи існують.
- int_least8_t, uint_least8_t; int_least16_t, uint_least16_t; int_least32_t, uint_least32_t; int_least64_t, uint_least64_t — знакові та беззнакові цілі типи величиною, найменшою із тих, які можуть вмістити типи вказаної ширини (8, 16, 32, 64 біти, відповідно).
- int_fast8_t, uint_fast8_t; int_fast16_t, uint_fast16_t; int_fast32_t, uint_fast32_t; int_fast64_t, uint_fast64_t — найшвидший із типів, що можуть вмістити вказану кількість біт.
- intptr_t, uintptr_t — цілий тип, здатний вмістити вказівник даної системи.

[13] Щодо проблем з ними, особливо в C: "A Special Kind of Hell - intmax_t in C and C++".

"Конкретизовані" floating point типи – C++23

- C++23 додав синоніми: std::float16_t, std::float32_t, std::float64_t, std::float128_t, std::bfloat16_t.
- Як і для цілих типів, це синоніми типів, що вже підтримуються компілятором – інакше відсутні.
- Їх оголошено в <stdfloat>.

- Наявність цих типів може залежати від конкретного компілятора.
- Константи, що містять найменші та найбільші значення цих типів, типу INT_LEAST8_MAX, знаходяться у цьому ж <cstdint>[14]. Аналогічні константи для фундаментальних типів знаходяться в файлі <climits>[15].
- C++ надає зручніший інструмент, ніж ці макроси – шаблон std::numeric_limits із <limits>.
- Детальніше про плюси і мінуси їх використання: `https://blog.feabhas.com/2024/01/embedded-expertise-beyond-fixed-size-integers-exploring-fast-and-least-types/`.
- `https://blog.feabhas.com/2024/02/navigating-memory-in-c-a-guide-to-using-stduintptr_t-for-address-handling/` – дещо непереконливий есей про uintptr_t.

[14] Детальніше див. `https://en.cppreference.com/w/cpp/header/cstdint`.

[15] `https://en.cppreference.com/w/cpp/header/climits`.

Проблеми цілих типів

- Порівняння знакових і беззнакових типів небезпечне – від'ємні значення будуть перетворені в беззнакові згідно доповнювального коду, тому, наприклад, "x > -1" для x типу unsigned поверне false майже завжди.
- `https://devblogs.microsoft.com/oldnewthing/20230619-00/?p=108355` – "*Why am I being told about a signed/unsigned comparison, and why only sometimes, and how can I fix it?*"

Тип void

Тип використовується для повідомлення про відсутність інформації про тип.

- Зрозуміло, що створювати змінні типу void не можна.
- Якщо функція "повертає" void — це значить, що вона не повертає жодного значення: `void somefn();`.
- "Сирий" вказівник, вказівник без типу — `void*`.

Конкретний вибір розміру типів називають моделлю даних (data model).
Широкого використання набули чотири моделі даних, які іноді записують у форматі sizeof(int)/sizeof(long)/sizeof(void*)[16]:

- LP32 чи 2/4/4 (int – 16 біт, long і pointer 32 біти). Win16 API, в певній інтерпретації – DOS.
- ILP32 – 4/4/4. Win32 API, 32-бітні POSIX-системи.
- LLP64 – 4/4/8. Win64 API.
- LP64 – 4/8/8. 64-бітні POSIX-системи.
- ILP64, 8/8/8, зараз не трапляється, використовувалося деякими ранніми (не x86) 64-бітними системами (згідно cppreference, Unicos on Cray).
- AVR8 використовує ціле сімейство систем, які можна охарактеризувати як 2/4/2, але з обмовками і нюансами.
- TMS320C28xx – приклад DSP, де байт та char – 16 біт: "MS320C28x Optimizing C/C++ Compiler User's Guide: 6.4 Data Types"

[16] Враховуючи, що всі вказівники мають однаковий розмір.

Оператор sizeof

Оператор **sizeof** повертає розмір типу чи змінної, вважаючи розмір char за одиницю. На більшості (але не на всіх!) системах це відповідає розміру в байтах.

Детальніше, якщо хочеться швидко отримати інформацію, див., зокрема, `http://en.wikipedia.org/wiki/Sizeof`.

Застосовувати його можна і до імен типів і до змінних:

```
int b = 4444;

cout << sizeof(b) << endl;
cout << sizeof(int) << endl;
// Або:
cout << sizeof b << endl;
// Для типів дужки обовязкові'
```

Стрічкові та символьні літерали

Літерал — запис фіксованого, заданого в тексті програми, значення.

- Символьні літерали — один символ, оточений одинарними лапками: `'a'`, `'C'`, `'%'`, `'\n'`, тощо. Задані таким способом літерали мають тип char[17].
- Стрічкові літерали — послідовність символів у лапках. Приклад: `"ABC"`. Розмір стрічкового літералу на одиницю більший, за кількість літер — для збереження символу кінця С-рядка, нульового символу: `sizeof("ABC")==4`. Може існувати порожній стрічковий літерал, `""` — між лапками порожньо, що відповідає відсутності символів (окрім нуля). Ці літерали мають тип char[][18].
- Стрічкові літерали, які йдуть один за одним, розділені хіба символами-роздільниками[19], об'єднуються:

```
cout << "abc"
        "DEF" << endl;
```

виведе `abcDEF`.

[17] Існують також літерали "довгого" символьного типу, wchar_t. Задаються вони якось так: L'ab', конкретні значення залежать від реалізації.

[18] Або, завдяки тому, що змінна-масив фактично є вказівником на його початок, та існуванню в C/C++ конструкцій виду `char *s="abc";`, тип char*.

[19] Інша назва – пробільні символи, це пробіли, табуляції, символами кінця рядка.

Керувальні послідовності — Escape sequences

- \n — Новий рядок.
- \t — Табуляція.
- \a — Гудок.
- \\ — Символ \.
- \' — Апостроф ("одинарні" лапки символьних літералів).
- \" — Подвійні лапки.
- \xHHH — Символ з шістнадцятковим кодом HHH. Кількість цифр не обмежена — кінцем вважається перший символ, який не є шістнадцятковою цифрою. Однак, отриманий таким чином код повинен вміщатися у відповідному символьному типі.
- \ooo — Символ з вісімковим кодом ooo. Кількість цифр у коді може бути 1, 2 або 3.
  - Жадібність керувальних послідовностей шістнадцяткових та вісімкових літералів може створювати проблеми, коли після \xHH має знаходитися звичайний символ, що відображає цифру або літери, які будуть інтерпретуватися, як шістандцяткові цифри.
  - З цим можна боротися за допомогою об'єднання стрічок: "\xAB""C".
- C++23 додав більш однозначні способи задати код літери: \o{} для вісімкових та \u{}, \x{} для шістнадцяткових.

Екзотичні керувальні послідовності

Додатково C++ знає наступні, рідко вживані, керувальні послідовності

- `\v` — Вертикальна табуляція.
- `\r` — Carriage return
- `\f` — Form feed
- `\?` — Знак запитання, "?"
- Цей список менш-більш вичерпний, в стандарті C++ інші відомі послідовності не підтримуються. Див. також `https://en.cppreference.com/w/cpp/language/escape`.

Стрічкові та символьні літерали в C++11 — 1

- C++11 включає два нових символьних типи: `char16_t` і `char32_t`. C++20 додав `char8_t`.
- Нульовим символом цих типів є `char16_t()` і `char32_t()`, відповідно.
- Завдяки ним, є можливість зберігати символи в Unicode кодуваннях UTF-8[20], UTF-16, UTF-32.
- Для представлення цих кодувань з'явилися відповідні стрічкові літерали:
  - `u8"This is a UTF-8 string."` — UTF-8, типу `const char[]` (як і "звичайні" C-стрічки).
  - `u"This is a UTF-16 string."` — UTF-16, типу `const char16_t[]`.
  - `U"This is a UTF-16 string."` — UTF-32, типу `const char32_t[]`.
- В такі літерали можна підставляти коди символів Unicode (Unicode codepoints)[21]:

```
u8"This is a Unicode Character: \u2018."
u"This is a bigger Unicode Character: \u2018."
U"This is a Unicode Character: \U00002018."
```

- Після `\` йде шістнадцятковий код символу, без звичних 0x перед ним. `\u` очікує 16-бітний код, рівно 4 цифри, `\U` — 32-бітний, 8 цифр (звідти й нулі в прикладі вище).

[20] Нагадаємо, що char гарантовано може зберігати один елемент кодування UTF-8.

[21] Приклад взято із вікі. Про символ U+2018: http://www.charbase.com/2018-unicode-left-single-quotation-mark.

Стрічкові та символьні літерали в C++11 — 2

- Ввід-вивід Unicode стрічок напряму iostream не підтримується, потрібна допомога шаблонного класу `wbuffer_convert<>/wstring_convert<>` та codecvt всіляких[22]:

```
char str_Utf8[] = u8"This is a UTF-8 string.";
char16_t str_Utf16[] = u"This is a UTF-16 string.";
char32_t str_Utf32[] = U"This is a UTF-32 string.";
cout << str_Utf8 << endl;
std::string temp;
std::wstring_convert<std::codecvt_utf8_utf16<char16_t>,
                     char16_t> myconv16_to_con;
temp = myconv16_to_con.to_bytes(str_Utf16);
cout << temp << endl;

std::wstring_convert<std::codecvt_utf8<char32_t>,
                     char32_t> myconv32_to_con;
temp = myconv32_to_con.to_bytes(str_Utf32);
cout << temp << endl;
```

Див. також std::u16string та std::u32string.

[22] Детальніше див. сам кінець глави 16.4.4, "Character Classification and Conversion" в книзі "The C++ Standard Library: Second Edition" від Nicolai M. Josuttis (сторінка 903). Визнано застарілим в C++17, викинуто в C++26, але поки без заміни: https://stackoverflow.com/a/42946556. На практиці – див. boost::locale із ICU в ролі бекенда. Щодо консолі Windows та Unicode – взагалі ціла окрема історія...

Стрічкові та символьні літерали в C++11 — 3

- C++11 також забороняє можливе раніше (C++98/03) небезпечне присвоєння стрічкових літералів вказівникам типу `char*` — дозволені лише `const char*`.
- Ще один новий тип стрічкових літералів – "сирі" (raw) літерали. Про них нижче.
- C++23 дозволив використання імен Unicode-символів: `'\N{LATIN CAPITAL LETTER A}'`. Див. також "C++23: Growing unicode support".
- З C++23 підтримка UTF8 обов'язкова.
- ◇ Деякі подробиці: "C++23: Encoding related changes".
- Оновлення щодо літералів у C++26: "C++26: Cleaning up string literals".

"Сирі" (raw) літерали C++11

- Є ситуації, коли для створення стрічок доводиться дуже активно використовувати керувальні послідовності, наприклад, коли пишуться літерали, що містять XML, команди скриптових мов, або[23] регулярні вирази.
- В C++11 додано спеціальні літерали, які дозволяють використовувати будь-які символи безпосередньо.
- Найпростіший їх варіант: все, що йде між R"(і)" стає частиною стрічки, ніяк при цьому не інтерпретується.
- Однак, що, якщо в стрічці треба оці "(чи)"? На такий випадок є розширений варіант: все, що починається із RXXX"(та)XXX", де XXX — довільна послідовність символів, довжиною до 16, яка лише не може містити пробілів, \, (,), інтерпретується як сирий літерал:

```
R"(This is "raw" \ string \n literal!  )"
R"delim(This is "raw" \ string \n literal with )"!  )delim"
```

- Можуть займати багато рядків.
- Сирі літерали цілком сумісні із u8-, u-, U-стрічками:

```
uR"(This is a "raw UTF-16" string.)"
```

[23] Ось де точно "страх и ужас"...

Цілі літерали

- Цілі літерали — спосіб задання цілих констант.
- Найпростіший їх різновид — десяткові літерали: **1, 523, 0, -177**. Якщо літерал не поміститься у змінній, призначеній для нього, компілятор видаватиме попередження.
- Існують також шістнадцяткові та вісімкові літерали.
- Шістнадцяткові літерали задаються за допомогою послідовності символів: **0x**: 0x5AF, 0x10, 0x3d. Літери A, B, C, D, E, F можна використовувати як великі, так і малі.
- Вісімковий літерал задається за допомогою нуля: **05, 0333, 0177**. **УВАГА!** Не ставте нуль перед цілим числом "просто так"! Це зробить його вісімковим.
- **C++14** ввів також двійкові цілі літерали: `0b010101010`. Крім того, GCC підтримував їх як розширення. (Див. також BOOST_BINARY: `http://www.boost.org/doc/libs/1_55_0/libs/utility/utility.htm#BOOST_BINARY`).
- В цілих літералах можна використовувати роздільник: ', він ігноруватиметься компілятором: 123'456'777.
- C++23 додав:
  - літерали для size_t – zu (чи uz), великі чи малі літери – не важливо.
- літерали для ptrdiff_t[24] – z (Z).

[24] В стандарті чомусь сказано дещо дивно: "the signed version of std::size_t (since C++23)", іншими словами, ptrdiff_t це стосується.

Типи цілих літералів

- Тип літералу можна вказати за допомогою спеціальних суфіксів. Якщо суфіксів немає, то для десяткових літералів використовується найменший із типів, що може вмістити значення: int, long int, unsigned long int. Вісімковий літерал після int спробує помістится в unsigned int, потім вже в long int.
- Суфікс **u** (або **U**) змушує вибирати тип із беззнакових. Буде вибрано unsigned int, або, якщо не поміститься в нього, unsigned long int.
- Суфікс **l** (або **L**) робить тип "довгим", тобто вибрано перший із типів, у який може помістится його значення: long int, unsigned long int.
- Суфікс **ll** (або **LL**) відповідає типу long long (C++11).
- Суфікс **ul**/**ull** (або будь яка із можливих комбінацій великих та малих u і l/ll) вказує, що літерал матиме тип unsigned long int/unsigned long long int.

Тобто, 5 буде константою типу int, 5U — unsigned int, 5L — long int, 5UL — unsigned long int.

Тип літералу, зокрема, впливає на вибір перевантажених функцій та операторів.

Літерали з рухомою комою

- Такі літерали задаються звичним[25] способом: **1.0**, **5.13**.
- Дозволяється опускати рівну нулю цілу частину: **0.345** повністю еквівалентно **.345**.
- Дозволяється опускати рівну нулю дробову частину: **2.0** еквівалентний **2.**.
- Дозволяється використовувати наукову нотацію: $1.23e4 = 1.23 \cdot 10^4$, $5.67E-11 = 5.67 \cdot 10^{-11}$.
- В науковому записі можна використовувати як E так і e, однак **заборонено використовувати пробіли**!
- Існують також екзотичні шістнадцяткові: "`0xA.BCp10`" (C++17). Експонента, яка відділяється p чи P – обов'язкова, її значення – десяткове число, тобто, "p10" означає $2^{10} = 1024$.

Типи літералів з рухомою комою:

- По замовчуванню використовується тип double.
- За допомогою суфікса f (або F) можна створити літерал типу float.
- Суфікс l чи L, застосований до літералу із рухомою комою, робить його long double.
- C++23 додав f16, f32, f64, f128, bf16, F16, F32, F64, F128, BF16.

[25] Для іноземців — використовується десяткова крапка.

Булеві літерали, **true** і **false**

У C++:

- 0 — false.
- Будь-яке інше значення — true! Тобто, і 1, і -376, і 'A' є "істиною".
- Однак, сама мова використовуватиме для true виключно значення 1.

Завдяки згаданому правилу відповідності, можна писати замість:

```
if( x==0 ) { ..... }
```

ось так:

```
if( !x ) { ..... }
```

Програмісти на C надають перевагу робити саме так, однак в програмах C++, навпаки, так робити не прийнято.

Булевий тип в C

Булеві значення в C з'явилися лише в стандарті C99. При чому, введені вони по іншому, ніж в C++:

- Замість типу bool використовується _Bool.
- true, false є макросами препроцесора із файлу `stdbool.h`.:

```
#define true    1
#define false   0
```

◇ Так зроблено, щоб не зламати старий код, який ці імена міг використовувати як ідентифікатори. Ім'я _Bool було й так зарезервованим, а кому потрібна ще й зручність – явно додає файл заголовків, якого раніше не було.

- Додатково оголошується константа:

```
#define __bool_true_false_are_defined   1
```

Користувацькі літерали C++11 — Draft

- C++ додав можливість створювати власні літерали для користувацьких класів.
- To be done. Example:

```
OutputType operator "" _suffix(const char * literal_string);

OutputType some_variable = 1234_suffix;
```

Користувацькі літерали стандартної бібліотеки – C++14+

- "abc"s відповідає літералу класу std::string – уникаючи перетворення із C-стрічки.
- `5s, 10m, 11h` тощо відповідають типу std::chrono::duration.
- `5.3i` відповідають уявному числу $0 + 5.3i$ типу std::complex<double>.
- C++17 додав string_view_literals із суфіксом sv.
- Оголошені такі літерали у своїх просторах імен, (std::string_literals, std::chrono_literals, std::complex_literals), але найпростіший спосіб доступитися до них це std::literals:

```
using namespace std:: literals ;

auto a_string = "Hello!"s; // type std :: string
auto a_minute = 60s;   // type std :: chrono :: duration = 60 seconds
auto a_day = 24h;      // type std :: chrono :: duration = 24 hours
auto pole = 3i;        // type std :: complex<double> = 0 + 3.0i
```

Оголошення — definitions

- Перш ніж використовувати, будь-яка сутність в програмі повинна бути оголошена.
- Оголошення вказує існування та призначення певного імені. Також воно може зв'язувати її із відповідною сутністю, тоді це буде зразу і визначенням (definition).
- Оголошувати можна змінні, типи, функції, структури та класи, простори імен, перерахування.
- Оголошень в програмі може бути скільки завгодно. Визначення повинне бути лише одне. Всі оголошення повинні узгоджуватися!

Структура оголошень

Оголошення складається із чотирьох частин. Деякі з них можуть бути відсутні.

```
int array [4]={5,1,2,3};
int b = 5;
int c;
extern int d;
char* names[] = {"Oleg", "Andriy","Taras"};
```

1. Специфікатор. Спеціальне ключове слово, що задає властивості, не пов'язані із типом. Наприклад, virtual, extern, volatile, static.
2. Базовий тип (включаючи константні, типу const int).
3. Частина, що власне оголошує — ім'я плюс можливі оператори оголошення.
4. Ініціалізатор, який задає початкове значення.

Завершується оголошення крапкою з комою.

Детальніше див.

https://en.cppreference.com/w/cpp/language/declarations.

- Оператори оголошення:
  - * — вказівник, префікс.
  - *const — константний вказівник, префікс.
  - & — посилання , префікс.
  - && — rvalue-посилання, префікс.
  - [] — масив, суфікс.
  - () — функція, суфікс.
- Суфікси вважаються тісніше пов'язаними із типом, ніж префікси, тому, наприклад: `type *var[]` — масив вказівників.
- Можна оголошувати декілька імен одним оголошенням:

  int a,b,c;

- **УВАГА!** Оператори оголошення стосуються тільки однієї змінної із декількох:

  int *a,b; // *a* ——— вказівник на *int, b* ——— змінна типу *int*

- **УВАГА!** Оголошення змінної без специфікатора extern, є зразу і її визначенням!

Ініціалізація

- Якщо задано ініціалізатор, він присвоюється змінній.
- Якщо його немає — глобальним змінним, змінним з просторів імен та статичним змінним (останні два види розглянемо в подальшому), присвоюється нуль або інше еквівалентне значення по замовчуванню. Всі інші — локальні[26] та динамічні змінні не ініціалізуються і можуть містити що завгодно.
  - ◇ Аналіз ініціалізації фундаментальних типів, до C++26 і після: `https://herbsutter.com/2024/08/07/reader-qa-what-does-it-mean-to-initialize-an-int/`.
- Об'єкти — екземпляри класів, ініціалізуються конструкторами відповідних класів. Детальніше їх буде розглянуто в подальшому.
- Ініціалізація в C++ – дуже складна тема. Існує навіть окрема книга: `https://leanpub.com/cppinitbook`, `https://www.cppstories.com/2022/cpp-init-book/`.

[26] Які також називаються автоматичними

Специфікатор

Специфікатори, типу virtual, будемо розглядати в подальшому, а поки перерахуємо ті, які впливають на властивості змінних :

- static — для **локальних** (автоматичних) змінних вказує, що змінна ініціалізується лише раз. Для **глобальних** змінних та функцій забороняє використовувати її з інших модулів. Для класів – такі члени спільні для всіх екземплярів класу (об'єктів).
- extern — змінна лише оголошується, визначення дано в іншому модулі.
- volatile — будь-які звертання до такої змінної вважати видимими ззовні – такими, що мають побічні ефекти. Наприклад, змінна може змінюватися з-зовні програми[27].
- З C++17 register зарезервовано, але не використовується.
- Слово auto[28] має нове значення в стандарті C++11 — автоматичне виведення типів. Детальніше про нього пізніше.
- Також пізніше будуть розглядатися всілякі constexpr, consteval. constinit, thread_local тощо.
- Трішки філософських міркувань про їх порядок: https://quuxplusone.github.io/blog/2021/04/03/static-constexpr-whittling-knife/.

[27] Подробиці див. https://blog.feabhas.com/2020/04/side-effects-and-sequence-points-why-volatile-matters/ та https://quuxplusone.github.io/blog/2022/01/28/volatile-means-it-really-happens/.

[28] Початковий зміст якого давно не використовувався.

Область видимості змінних

- Змінною можна користуватися від моменту її оголошення, до кінця блоку.
- В даному блоці змінна із конкретним іменем може бути оголошена лише раз.
- Якщо в блоці є вкладені блоки, змінні, оголошені перед таким вкладеним блоком, видимі у ньому.
- Однак, у вкладеному блоці можна визначати змінні із іменами, використаними у блоках вищого рівня. При цьому така змінна буде повністю заміщати оголошення у блоках вищого рівня.
- Змінні, оголошені за межами функцій, включаючи функцію main, називають глобальними. Їх область видимості — глобальна, звертатися до них можна із будь-яких функцій, оголошених після них.
- Звернутися до затінених глобальних змінних можна за допомогою конструкції: `::var_name`. Звернутися до затінених локальних змінних неможливо.

Область видимості – приклад

```
int global_var=5;
int main() {
    cout << "global_var = " << global_var << endl;

    // Наступний рядок помилковий! local_var1 ще не визначена.
    // cout << "first local variable = " << local_var1 << endl;

    char local_var1='A';
    int some_number=11;

    // Наступний рядок помилковий! Ім'я local_var1 вже зайняте.
    //int local_var1=22;

    double i=5.5;
    cout << "i before block : " << i << endl;

    { // Блок
        cout << "i at the block start : " << i << endl;
        int i=17;
        cout << "i after redefinition : " << i << endl;
    }
    cout << "i after the block: " << i << endl;

    return 0;
}
```

→ Ця термінологія дещо застаріла, в сучасному C++ використовується більш просунута[29], однак почати варто з неї – розмова про rvalue reference і компанію – помітно громіздкіша і буде далі.

- lvalue — збірна назва для того чи іншого посилання на об'єкт[30] в пам'яті.
- Вихідним значенням lvalue було "щось, що може стояти ліворуч від оператора присвоєння".
  - ◇ Однак, в сучасному C++ існують і lvalue, яким не можна присвоювати, наприклад – посилання на константу.
  - ◇ Тому, правильніше казати, lvalue – сутність, яка існує і за межами поточного виразу, така, для якої ви маєте або ім'я, або вказівник на неї.
- rvalue — все що не є lvalue[31]. Це може бути літерал-значення, тимчасова змінна, сконструйована під час виклику функції чи повернення з неї, тощо.

---

[29] Див. `https://en.cppreference.com/w/cpp/language/value_category`.

[30] Слово об'єкт вжито в розумінні певної сутності, а не об'єктів як екземплярів якогось класу.

[31] Іншими словами — конкретне значення, наприклад літерал, який використовується для ініціалізації.

Константи

- Писати в програмі магічні числа, типу 1917, 33, 3.1415192 — погана практика. Важко змінювати, якщо така потреба виникла, важко знати, котрі із однакових значень відображають одну із різних, але з рівними значенням, сутностей.
- Зрозуміло, що використовувати змінну для кожного числа нераціонально, небезпечно і, чисто технічно, не завжди можливо.
- C++ містить поняття константи — величини певного типу, яку не можна змінювати. Їх значення задається при створенні.
- Для створення константи використовується ключове слово const:

```
const int doors1 = 2; // Константа, число
int const doors2 = 2; // Синонім до попереднього:
                      // положення const відносно типу не важливе
const int arr[4] = {1,3,5,7}; //Всі елементи масиву --- константи
const int *ptr;     // Вказівник на константні дані
int a=5;
int * const ptr = &a;    // Константний вказівник
                         // на дані, які можна змінювати
const int * const ptr = &a;    // Константний вказівник
                               // на дані, які заборонено змінюва-
```

- Константність варто використовувати широко – але не надміру широко.
  - ◇ Про застереження: `https://quuxplusone.github.io/blog/2022/01/23/dont-const-all-the-things/`. Безглуздо повертати константні значення (не плутати із поверненням константних посилань), передавати константні значення чи робити константами локальні змінні, крім особливих випадків.
- Немає сенсу, і часто – шкідливо, повертати константі значення з функцій.
  - ◇ Константні посилання часто мають сенс.
  - ◇ "Why don't compilers warn for 'const T f()'?" – есей на тему.

Константи засобами препроцесора

Існує також застарілий, але іноді необхідний, спосіб оголошення константи:

```
#define SOME_CONSTANT 1000
```

призведе до того, що всюди в тексті програми, перед компіляцією, `SOME_CONSTANT` буде замінено на 1000.

Так як такий спосіб повністю ігнорує структуру програми на C++, виконуючи примітивну текстову заміну ідентифікаторів[32], слід максимально його уникати.

[32] Вміння розпізнати ідентифікатори — одне із дуже небагатьох знань препроцесора про синтаксис C++.

Перерахування — enumerations

Ще один похідний тип, значення змінних якого обмежено списком констант

enum COLOR {RED, BLUE, GREEN, WHITE, BLACK};

- COLOR — нове ім'я типу, після оголошення його можна використовувати для створення — оголошення, змінних.

```
COLOR sky_color;
sky_color = BLUE;
sky_color = WHITE;
```

- RED, BLUE, GREEN, WHITE, BLACK — нові константи в області видимості COLOR[33].
- По замовчуванню перша має значення 0, друга 1, і т.д.
- Це можна змінити:

enum COLOR {RED=100, BLUE, GREEN, WHITE, BLACK};

- тепер RED=100, BLUE=101 і т.д. Можна навіть так:

enum COLOR {RED, BLUE, GREEN=200, WHITE, BLACK=300};

- Тоді RED = 0, BLUE=1, GREEN=200, WHITE=201, BLACK=300.

[33] Якщо enum оголошений в класі/структурі, до до цих констант можна звертатися за допомогою ".", "->", "::".

- Константи-значення перерахування оголошуються в тій же області видимості, що й перерахування. Тому, з одного боку, присвоюючи:

```
COLOR x;
x = RED;
```

немає потреби вказувати, якому перерахуванню належить ідентифікатор RED. З іншого, засмічується область видимості, зокрема, різні перерахування в одній області видимості не можуть мати однакових значень.

- Дозволено присвоювати значення типу перерахувань цілим змінним, але не навпаки[34]:

```
COLOR x = RED;
int y;
y = x; // OK
//x = 5; // Error!
```

[34] Майте на увазі, в чистому C таке перетворення теж дозволене.

Припустимі значення змінних злічених типів – до C++11

- Фізично перерахування зберігається у чомусь типу unsigned int. Подробиці[35] залежать від платформи і багатьох нюансів.
- Якщо тип для збереження не вказано, припустимими є:
  - Явно перераховані значення:

```
enum BITS{BIT0=1, BIT1=1<<1, BIT2=1<<2, BIT3=1<<3};
BITS b1, b2, b3;
b1 = BIT0;
```

  - Допускаються їх комбінації, які не виходять за межі кількості біт, потрібних, щоб представити найбільше із значень:

```
b2 = static_cast<BITS>(BIT1 | BIT3);
b3 = static_cast<BITS>(BIT1 + BIT3);
```

    Явне перетворення типів, static_cast, необхідне, так як результат виразів типу "BIT1 + BIT3" є цілим числом, а не одним із "офіційних" значень BITS.
  - А це unspecified-поведінка до C++17, undefined-поведінка в C++17 і далі:

```
b3 = static_cast<BITS>(BIT3 + BIT3);
```

[35] Див. розділ 4.8 книги Бьёрн Страуструп, "Язык программирования C++", специальное издание. Для enum class з C++11 дозволено вказувати, яким має бути базовий тип.

Припустимі значення змінних злічених типів – C++11

- C++11 дозволив конкретизувати, який тип потрібно використати для збереження:

  enum reg_val: **unsigned int** { ..... };

- У фігурних дужках може нічого й не бути. Також, завдяки цій конструкції, можна лише оголосити enum, без визначення:

  enum reg_val: **unsigned int**;

- Якщо вказано тип для збереження, enum може містити будь-які його значення.

Строго типізовані перерахування C++11 – scoped enumerations

- C++11 вводить новий вид перерахувань — "enum class"[36]:

  enum class Enum2 : **unsigned int** {Val1, Val2};

- Значення констант enum class не виносяться в область видимості, що охоплює оголошення таких перерахувань.
- ◇ Звертатися до них можна так:

  Enum2 ev = Enum2::Val2;

- ◇ Цей же запис було ретроактивно дозволено для звичайних enum.
- C++20 дозволив притягувати такі імена в поточну область видимості:

```
void fn(){
    using enum Enum2;
    Enum2 ev = Val2;
}
```

[36] Запис enum struct є повним синонімом.

Строго типізовані перерахування C++11 – scoped enumerations

- Для scoped enumerations заборонено змішування із цілими типами.
- ◇ За одним винятком, починаючи із C++17 їх можна ініціалізувати цілим значенням із використанням списку ініціалізації з одним елементом:

```
enum byte : unsigned char {};  // std::byte definition for C++17
byte b { 42 };    // OK -- C++17
byte c = { 42 }; // error
byte e { -1 };    // error
```

- В інших випадках потрібно використовувати все той же static_cast.
- Есей про (незручне) використання їх для обробки помилок: "C++ Enum Class and Error Codes"[37].

[37] Див. також продовження з дискусією: "part 2". "Третя частина зосереджується на тому, що виключення, насправді, зручний метод, особливо за сучасних компіляторів і апаратного забезпечення.

Using для scoped enumerations (C++20)

- C++20 додав можливість притягувати імена з строго типізованих просторів імен в поточну область видимості:

```
enum class Enum2 : unsigned int {Val1, Val2};
enum class Enum3 : unsigned int {Val3, Val4};
   .....
using Enum2;
using Enum3::Val4;
x = Val1;
y = Val2;
z = Val4
k = Enum3::Val3; // А ось "k = Val3" буде помилкою.
```

- Як видно із прикладу, можна вносити як всі імена з перерахування, так і лише окремі із них.
- Можна застосовувати і до "просто" enum, але особливого сенсу так робити немає.
- Використовувати потрібно максимально локально. Корисні, наприклад, у switch по елементах перерахувань.

Атрибути перерахувань (C++11)

- До перерахувань чи їх елементів можна застосовувати атрибути:

```
enum class en1{v1 [[nodiscard ]], v2 [[ deprecated ]]};
enum class [[deprecated ]] en2{v1, v2};

enum en3{v1 [[nodiscard ]], v2 [[ deprecated ]]};
enum [[deprecated]] en4{v3, v4};
```

std::underlying_type (C++11)

- `std::underlying_type<T>::type` – тип, який використовується для збереження перерахування T.
- `std::underlying_type_t` – синонім із C++14.
- В C++23 додали:
  - функцію перетворення типу `std::to_underlying()`.
  - трейт `std::is_scoped_enum_v`.

Обмеження неявних перетворень перерахувань – C++20-C++26

- В C++20 оголошено застарілими порівняння enum та floating point.
- В C++26 запропоновано заборонити також порівняння елементів різних enum.
- Відповідний документ: "Remove Deprecated Arithmetic Conversion on Enumerations From" .

Посилання

- https://shafik.github.io/c++/2022/11/12/enums-in-cpp-choice-oft-beguilded-part-1.html,
- https://shafik.github.io/c++/2022/11/15/exploring-clang-enum-impl-and-ub-part-2.html.
- https://www.cppstories.com/2024/enum-improvements/

typedef та using (C++11) – синоніми типів

- Є можливість створювати синоніми для типів.
- Для цього використовується ключове слово typedef:

```
typedef unsigned short int  ushort;
typedef vector<double>::const_iterator  vditerator;

ushort  var117;
vditerator  i;
```

  Іноді це зручно.

- В C++11 з'явилася ще одна форма, із використанням using. Рядки нижче, попарно синонімічні:

```
typedef vector<double>::iterator  titr;
using  titr  = vector<double>::iterator;

typedef double (*fnPtrT)(double, int);
using fnPtrT = double (double, int);
```

◇ Якщо ваш компілятор дозволяє, краще використовувати таку форму.

Запозичена із C множина типів не володіє строгою типізацією — вона допускає перетворення типів.
Неявно здійснюються наступні види/способи перетворень:

- Просування типів (types promotion) — під час арифметичних операцій, типи з менш ємких перетворюються у int або unsigned int, float в double:

```
int x,y;     short z;     char t;
x = y + z + t;
```

  Цікаві коментарі див., наприклад, тут:
  `http://www.idryman.org/blog/2012/11/21/integer-promotion/`
  Ціль такого перетворення — працювати із максимально природними для апаратури типами.

- Арифметичні перетворення. Якщо просування було недостатнім для перетворення всіх частин виразу до одного типу, компілятор виконує, в рамках виразу, перетворення до першого достатньо загального типу, згідно наступного списку пріоритетів:

```
int ==> unsigned int ==> long ==> unsigned long ==>
        long long ==> unsigned long long ==> float ==>
        double ==> long double
```

- Під час присвоєння результатів різних арифметичних типів, відбувається відповідне неявне перетворення. Якщо значення більшого за розміром типу вміщається в змінній меншого – воно буде коректно присвоєно. Якщо ні, результат залежить від платформи (implementation defined):

```
int x=5;
short z;
z = x;
```

- Аналогічно, дозволені перетворення між цілими числами та числами з рухомою крапкою:

```
int x;
double d;
x = d;
d = x;
```

Ціле значення, якщо це можливо, точно перетворюється в double, чи інший тип з рухомою крапкою. Під час зворотного перетворення, **дробова частина відкидається** — ніякого заокруглення не відбувається. Зауважте, що втрата інформації може відбуватися і при перетворенні цілого в число з рухомою комою — якщо в ньому забракне значущих цифр.

- Якщо беззнаковому типу присвоюється від'ємне значення, воно зберігається як послідовність біт, яка представляє від'ємне число у доповнювальному коді[38].
- Логічне значення перетворюється в цілі наступним чином: false перетворюється в 0, true – в 1.
- Зворотне перетворення — будь-яке не нульове значення це true, 0 — false.
- Правила перетворення вказівників будуть розглядатися окремо. Для повноти викладу: 0 і nullptr (а також, макрос NULL[39]) є сумісним із будь-якими типами вказівників. Будь який вказівник може бути неявно перетворено на `void*`.

[38] Що важливо, при цьому стандарт не вимагає, щоб від'ємні числа представлялися саме у доповнювальному коді, хоч так робить переважна більшість платформ. Вимога стосується лише перетворення знакових величин у беззнакові.

[39] Вигляд макроса стандартом не фіксується. В C++ він підставлятиме літерал 0 або nullptr. В C часто використовується 0, перетворених до void*: "#define NULL ((void*)0)", однак – через суворішу систему типів, в C++ це заборонено.

Обережно із перетвореннями!

- Робимо такі присвоєння та дивимося на результат[40]:

```
size_t a;
a = -1;          <print a>
a = -1u;         <print a>
a = -1lu;        <print a>
```

- Вивід (вказано розмір int/long/long long/size_t):

```
[Win32, Microsoft Visual C++ version 14.1, 64-bits, 4i/4l/8ll/8z]
0 FFFF FFFF FFFF FFFF # -1
0 0000 0000 FFFF FFFF # -1u
0 0000 0000 FFFF FFFF # -1lu

[Win32, Microsoft Visual C++ version 14.1, 32-bits, 4i/4l/8ll/4z]
0 0000 0000 FFFF FFFF # -1
0 0000 0000 FFFF FFFF # -1u
0 0000 0000 FFFF FFFF # -1lu

[linux, GNU C++ version 9.3.0, 64-bits, 4i/8l/8ll/8z]
0 FFFF FFFF FFFF FFFF # -1
0 0000 0000 FFFF FFFF # -1u
0 FFFF FFFF FFFF FFFF # -1lu
```

[40] Форматування виводу – ширина 17 символів, HEX, uppercase. Пробіли розставлені вручну. Код – див. проект l01_dangerous_casts.

- ```
  [Win32, GNU C++ version 10.2.0, 64-bits, 4i/4l/8ll/8z]
  0 FFFF FFFF FFFF FFFF # -1
  0 0000 0000 FFFF FFFF # -1u
  0 0000 0000 FFFF FFFF # -1lu

  [linux, GNU C++ version 9.3.0, 32-bits, 4i/4l/8ll/4z]
  0 0000 0000 FFFF FFFF # -1
  0 0000 0000 FFFF FFFF # -1u
  0 0000 0000 FFFF FFFF # -1lu
  ```
- Ще неочевидні приклади: `https://shafik.github.io/c++/2021/12/30/usual_arithmetic_confusions.html`.
  ```

- std::common_type із C++11 дозволяє знайти спільний тип – той, до якого приводяться її аргументи:

```
template <class T>
struct Number { T n; };

template <class T, class U>
Number<typename std::common_type<T, U>::type>
        operator+(const Number<T>& lhs, const Number<U>& rhs)
{
    return {lhs.n + rhs.n};
}
```

- Для арифметичних типів це щось, як тип виразу: T0() + T1() + ... + Tn().
- Детальніше: `https://en.cppreference.com/w/cpp/types/common_type`.

C++20

- C++20 додав кілька корисних для вирішення проблем із неявними перетвореннями цілих типів засобів.
- Функція std::ssize() повертає розмір контейнера як ptrdiff_t, дозволяючи уникнути змішування знакових та беззнакових типів.
- Функції std::cmp_equal, std::cmp_not_equal, std::cmp_less, std::cmp_greater, std::cmp_less_equal, std::cmp_greater_equal, які порівнюють значення, не перетворюючи їх перед тим – порівняння -1 (тип int) із 0u буде невірним за допомогою оператора: -1 > 0u == true; через перетворення типів, описані вище, а std::cmp_greater(-1, 0u) == true – дасть (математично) правильну відповідь.
- Функція:

  template< **class** R, **class** T >
  constexpr **bool** in_range(T t) noexcept

  дозволяє перевірити, чи t може бути перетворена до типу R без втрат.

Діаграма перетворень фундаментальних типів від hackingcpp.com

C++'s "Usual Arithmetic Conversions"

h/cpp hackingcpp.com

Original Purpose (goes back to C)

determine a common type for both operands of a binary operation ⊗ and its result

Example:

```
12          + 3.5
int(12)     + double(3.5)
double(12) + double(3.5) → double(15.5)
```

conversion

Operand A	⊗	Operand B (and vice versa)		
long double	⊗	double, float, any *INTEGER*	→	long double
double	⊗	float, any *INTEGER*	→	double
float	⊗	any *INTEGER*	→	float
INTEGER-TYPE1	⊗	*INTEGER-TYPE2*		1. Promotion of Both Operands 2. Conversion

Integer Conversion Rank $\mathcal{R}$

Standard Integral Types

smaller

$\mathcal{R}$(bool)
< $\mathcal{R}$(char) = $\mathcal{R}$(signed char) = $\mathcal{R}$(unsigned char)
< $\mathcal{R}$(short int) = $\mathcal{R}$(unsigned short int)
< $\mathcal{R}$(int) = $\mathcal{R}$(unsigned int)
< $\mathcal{R}$(long int) = $\mathcal{R}$(unsigned long int)

larger < $\mathcal{R}$(long long int) = $\mathcal{R}$(unsigned long long int)

$\mathcal{R}$(charXX_t) = $\mathcal{R}$(*underlying-int-type*(charXX_t))

General Rules

- $\mathcal{R}$(*T1*) ≠ $\mathcal{R}$(*T2*) even if Representation(*T1*) = Representation(*T2*)
- $\mathcal{R}$(smaller width) < $\mathcal{R}$(larger width)
- $\mathcal{R}$(signed *INTEGER-TYPE*) = $\mathcal{R}$(unsigned *INTEGER-TYPE*)
- if $\mathcal{R}$ (T1) > $\mathcal{R}$ (T2) and $\mathcal{R}$ (T2) > $\mathcal{R}$ (T3) then $\mathcal{R}$ (T1) > $\mathcal{R}$ (T3)
- $\mathcal{R}$ (*extended-int-type*) < $\mathcal{R}$ (*any-standard-int-type*)
- $\mathcal{R}$ (*extended-int-T1*) < impl. defined > $\mathcal{R}$ (*extended-int-T2*)

Surprising Examples

Unexpected Promotions

```
uint16_t(1) - uint16_t(2)
  ↳ int(1) - int(2)   (promotion)
  ↳ Result: -1

uint32_t(1) - uint32_t(2)
  ↳ no promotion   (both have R(int))
  ↳ Result: 4294967295
```

Platform Dependence

```
-3L < 3U
long int   unsigned int
```

width(long int) = width(unsigned int)
(e.g., both 32 bits on Windows platforms)

```
-3L < 3U
(2^32 - 3)U < 3U
4294967293U < 3U
⇒ false
```

(conversion)

width(long int) > width(unsigned int)
(e.g., 64 bits vs. 32 bits)

```
-3L < 3U
-3L < 3L
⇒ true
```

(conversion)

Integer Promotion

if $\mathcal{Rank}$(TYPE(Operand)) < $\mathcal{Rank}$(int)

TYPE(Operand)	→	Promoted To Type
signed char, unsigned char, signed short int, unsigned short int		int if it can represent all values of operand's type, unsigned int otherwise
char8_t, char16_t, char32_t, wchar_t		first type that can represent all values: int, unsigned int, long int, unsigned long int, long long int, unsigned long long int

Integer Conversion if *T1* ≠ *T2*

Operand A	⊗	Operand B (and vice versa)	
signed *INTEGER-T1*	⊗	signed *INTEGER-T2*	B converted to TYPE(A) if $\mathcal{R}$(TYPE(A)) > $\mathcal{R}$(TYPE(B))
unsigned *INTEGER-T1*	⊗	unsigned *INTEGER-T2*	B converted to TYPE(A) if $\mathcal{R}$(TYPE(A)) > $\mathcal{R}$(TYPE(B))
unsigned *INTEGER-T1*	⊗	signed *INTEGER-T2*	▪ B converted to TYPE(A) if $\mathcal{R}$(TYPE(A)) ≥ $\mathcal{R}$(TYPE(B)) ▪ A converted to TYPE(B) if TYPE(B) can represent all values of TYPE(A) ▪ otherwise, both A and B are converted to unsigned *INTEGER-T2*

v1.0

- Взято тут.

- `https://itnext.io/daily-bit-e-of-c-numbers-are-not-easy-74431416e754` – "Daily bit(e) of C++ : Numbers are not easy" – про перетворення фундаментальних типів, помилки з цим пов'язані та засоби із C++20 для боротьби із ними.
- `https://www.foonathan.net/2022/09/new-integer-types/` – цікава дискусія про варіанти цілих типів, які хотілося б бачити в стандарті.